

Class Practice Exercise 1.2

Data Analytics: Python Statistics Library | Problem-Solving Edition

IDE: Visual Studio Code | File: analytics_practice_1_2.py | Python 3.x

10

Problems to Solve

3

Difficulty Levels

8+

Stat Functions

Solutions

Included at End

Part A

File Setup in VS Code

Create your Python file before writing any code. All 10 exercises belong in a single file.

VS Code

File > New File > Ctrl+S (Cmd+S on Mac) > name it: **analytics_practice_1_2.py**

Confirm Python appears in the bottom-right status bar before writing any code.

Add this header at the very top of your file:

```
# =====  
# Class Practice Exercise 1.2 -- Data Analytics  
# Name: _____  
# Date: _____  
# Course: _____  
# File: analytics_practice_1_2.py  
# Topic: Python statistics library & descriptive stats  
# =====
```

SCREENSHOT A

Take a screenshot showing VS Code with: (1) the file tab named analytics_practice_1_2.py, (2) the complete header comment visible, and (3) Python in the bottom status bar. Paste below.

Part B

Key Concepts Reference

Study this reference table before starting. Return to it whenever you need a definition.

Concept	What It Is	Why Analysts Use It
Standard Library	Built into Python — no pip needed. Includes statistics, math, csv, datetime.	Lets analysts compute stats and parse files without any installation.
Third-Party Library	External packages installed via pip (pandas, numpy, matplotlib).	Core professional tools: larger datasets, richer visualizations, DataFrames.
import statement	Loads a module so its functions become available in your script.	Every analytics script starts here. Knowing what you import makes code intentional.

statistics module	Standard library module providing mean, median, mode, stdev, variance.	Quick descriptive analysis of small datasets without any setup.
Mean	Sum of all values divided by count. Sensitive to outliers.	Used for symmetric data. Can mislead when extreme values skew the average.
Median	Middle value when data is sorted. Resistant to outliers.	Preferred for skewed data (income, house prices, order values).
Mode	Most frequently occurring value. Can return multiple values.	Useful for categorical data or finding the most common measurement.
Standard Deviation	Average distance of each value from the mean.	Quantifies variability — essential for comparing consistency across groups.
Variance	Square of the standard deviation. Units are squared.	Foundation for advanced statistics; rarely reported directly to stakeholders.

Part C

10 Data Analytics Problems — Write Your Own Code

Instructions

For each problem: read the scenario, write your own solution from scratch, run it, and verify the output matches exactly. Add all exercises to `analytics_practice_1_2.py` separated by comment labels. Save with Ctrl+S and run with Ctrl+F5.

Ex. 1 Exploring Import Styles

[Beginner]

Analyst Scenario: *You just joined a data team. Before touching any data, you need to prove you can load Python's built-in tools three different ways.*

Your Task

1. Import the statistics module using a full module import, then print `math.pi` using the `math` module.
2. Re-import only the mean and median functions using a `from...import` statement.
3. Re-import the statistics module using the alias `stats`.
4. Using all three styles, compute and print the mean of `[10, 20, 30]` — once per style — and confirm all three print the same result.
5. Print the result of `math.sqrt(144)` using the full `math` module import.

Hint: Use `import statistics`, `from statistics import mean`, and `import statistics as stats`. Access `math.pi` with `math.pi`. String multiplication with `"#"` works for simple visual separators.

Expected Output

```
Exercise 1 -- Import Styles
math.pi = 3.141592653589793
sqrt(144) = 12.0
Mean (full module):      20
Mean (from import):      20
Mean (alias stats):      20
All three return the same result -- different styles, same function!
```

Write your code here:

**SCREENSHOT
1**

Run your file. Screenshot the terminal output for Exercise 1 showing all print lines. Paste below.

Analyst Reflection

Q1 What is the difference between import statistics and from statistics import mean? When would you choose each style in a real project?

Answer:

Q2 Why might a data analyst alias a library (e.g., import statistics as stats)? What are the trade-offs of using aliases?

Answer:

Q3 Name two standard library modules and two third-party libraries a data analyst would commonly use. What does each one do?

Answer:

Ex. 2 Central Tendency — Monthly Sales

[Beginner]

Analyst Scenario: Your manager handed you 12 months of sales figures and wants a one-line performance summary using the three measures of central tendency.

Your Task

1. Create a list: `monthly_sales = [42, 55, 61, 48, 73, 58, 66, 51, 70, 63, 55, 80]`.
2. Import the statistics module and calculate mean, median, and mode of the list.
3. Print the dataset, the count of months, and all three statistics with descriptive labels using f-strings.

4. Add an if/elif/else block: if mean > median, print "Insight: Mean > Median -- a high-sales month is pulling the average up." Apply the correct message for each case.

Hint: statistics.mean(), statistics.median(), statistics.mode() all accept a list. Use len() for the count. F-strings: f"Mean: \${sales_mean}k". Compare sales_mean and sales_median with > and <.

Expected Output

```
Exercise 2 -- Sales Central Tendency
Dataset: [42, 55, 61, 48, 73, 58, 66, 51, 70, 63, 55, 80]
Count:   12 months
Mean:    $60.17k (average monthly sales)
Median:  $59.5k  (middle-ranked month)
Mode:    $55k    (most frequent sales figure)
Insight: Mean > Median -- a high-sales month is pulling the average up.
```

Write your code here:

SCREENSHOT 2

Screenshot the terminal showing the Exercise 2 output including the Insight line. Paste below.

Analyst Reflection

Q1 What does the mean tell you that the median does not, and vice versa? In a business context, when is each more useful?

Answer:

Q2 The mode for this dataset is 55. What does that actually mean about the sales data? Should your manager care about the mode here?

Answer:

Q3 If January's sales were \$5k instead of \$42k, how would that affect the mean, median, and mode? Which statistic is most affected and why?

Answer:

Ex. 3 Outlier Effect — Employee Salaries

[Beginner]

Analyst Scenario: HR asks you to report the "typical" salary. One executive earns 10x the average employee. You must show your manager exactly how the outlier distorts the numbers.

Your Task

1. Create `salaries_no_outlier = [45000, 48000, 52000, 47000, 51000, 49000, 53000]` and `salaries_with_outlier` — the same list with 420000 appended.
2. Print mean and median for both datasets under separate headings ("--- Without Executive ---" and "--- With Executive ---").
3. Calculate the difference the outlier makes to the mean, and separately to the median. Print both differences.
4. Print a final conclusion: "Analyst Decision: Report the MEDIAN when outliers are present."

Hint: Use `{:,0f}` inside f-strings to add commas to large numbers (e.g., `f"${value:~,0f}"`). Subtract the two means to find the mean difference. Do the same for medians.

Expected Output

```
Exercise 3 -- Outlier Effect on Central Tendency

--- Without Executive Salary ---
Mean:    $49,286
Median:  $49,000

--- With Executive Salary ---
Mean:    $95,625
Median:  $50,000

Adding the outlier raised the mean by:    $46,339
Adding the outlier changed the median by: $1,000

Analyst Decision: Report the MEDIAN when outliers are present.
```

Write your code here:

SCREENSHOT
3

Screenshot the terminal showing both salary tables and the outlier impact amounts. Paste below.

Analyst Reflection

Q1 By how many dollars did the outlier change the mean vs the median? What does this tell you about which measure to trust?

Answer:

Q2 What would be the real-world consequences of reporting the mean salary as "typical" in an HR scenario where an executive outlier exists?

Answer:

Q3 Describe another real-world dataset where one outlier would severely distort the mean. What is the dataset, and what would the outlier be?

Answer:

Ex. 4 Mode Analysis — Customer Survey

[Beginner]

Analyst Scenario: You ran a 1–5 star customer satisfaction survey. Marketing wants the most common rating and a visual frequency breakdown.

Your Task

1. Create ratings = [4, 5, 3, 4, 5, 4, 2, 4, 5, 3, 4, 1, 4, 5, 3, 4, 4, 5, 2, 4] and sizes_ordered = ["M", "L", "M", "S", "M", "XL", "L", "M", "S", "M", "L", "M"].
2. Print the mode of ratings and the total response count.
3. Write a for loop over [1, 2, 3, 4, 5]. Inside the loop, count each rating's occurrences and print a text bar made of "#" characters — one "#" per response.
4. Print the mode of sizes_ordered (categorical data) and use statistics.multimode() on ratings to show all tied modes.

Hint: list.count(value) counts occurrences. "#" * n repeats the "#" character n times. statistics.mode() returns one value; statistics.multimode() returns a list of all most-common values.

Expected Output

```
Exercise 4 -- Mode Analysis
Most common rating: 4 stars
Total responses: 20
```

```
1 stars: # (1 responses)
2 stars: ## (2 responses)
3 stars: ### (3 responses)
4 stars: ##### (10 responses)
5 stars: #### (4 responses)
```

```
Most ordered size: M
```

All modes (ties included): [4]

Write your code here:

SCREENSHOT 4

Screenshot the terminal showing the bar chart and all mode output. Paste below.

Analyst Reflection

Q1 What does the mode tell marketing about customer satisfaction that the mean rating would not? Give one specific business decision you could make using each.

Answer:

Q2 Why is mode useful for categorical data (like sizes or colors) while mean is not? What would happen if you called `statistics.mean()` on `sizes_ordered`?

Answer:

Q3 What is the difference between `statistics.mode()` and `statistics.multimode()`? When would `multimode()` be essential in a real dataset?

Answer:

Ex. 5 Standard Deviation — Warehouse Temperatures

[Intermediate]

Analyst Scenario: *You manage two warehouses with the same average temperature, yet one has far higher product spoilage. You need to prove why using standard deviation.*

Your Task

1. Create `warehouse_A` = [68, 70, 69, 71, 68, 70, 69, 70, 68, 71] and `warehouse_B` = [58, 82, 61, 79, 55, 83, 64, 78, 60, 80].

2. Write a for loop that processes both warehouses as (name, data) tuples. Inside the loop, calculate and print: mean, median, standard deviation, variance, and range (max minus min).
3. After the loop, compute the ratio of Warehouse B's stdev to Warehouse A's stdev. Print: "Warehouse B has X.Xx more temperature variability than A." and a one-sentence business conclusion.

Hint: Use `statistics.stdev()` and `statistics.variance()`. `Range = max(data) - min(data)`. The ratio is `stdev_B / stdev_A`. Format to 1 decimal place with `:.1f`.

Expected Output

Exercise 5 -- Standard Deviation & Variance

```
--- Warehouse A ---
Temperatures: [68, 70, 69, 71, 68, 70, 69, 70, 68, 71]
Mean:        69.4 F
Std Dev:     1.07 F
Variance:    1.16
Range:       3 F (min=68, max=71)

--- Warehouse B ---
Temperatures: [58, 82, 61, 79, 55, 83, 64, 78, 60, 80]
Mean:        70.0 F
Std Dev:     10.97 F
Variance:    120.44
Range:       28 F (min=55, max=83)
```

Warehouse B has 10.3x more variability -- same average, very different spread.

Write your code here:

SCREENSHOT 5

Screenshot the terminal showing both warehouse comparisons and the variability ratio. Paste below.

Analyst Reflection

- Q1** Both warehouses have the same mean. How does standard deviation reveal the real difference? What business decision should you make based on this data?

Answer:

- Q2** Explain standard deviation in plain English to someone who has never taken a statistics class. Use the warehouse temperatures as your example.

Answer:

Q3

Why is variance always larger than standard deviation for the same dataset? In what situation would an analyst report variance instead of standard deviation?

Answer:

Ex. 6

Range & Spread — Website Traffic

[Intermediate]

Analyst Scenario: The marketing team wants to know: best day, worst day, typical daily traffic, and which days were statistical outliers over the past 30 days.

Your Task

1. Create `page_views =`
[1203,987,1450,1102,876,2341,1567,1234,905,1678,1345,2100,1456,890,1234,1567,1890,1023,1345,1678,934,1123,1456,1789,1012,1345,1567,1890,1234,2456].
2. Print: best day (max), worst day (min), range, mean, median, and std dev — all formatted with commas.
3. Use list comprehensions to count days above average and days below average. Print each with a percentage of the 30-day total.
4. Identify outlier days — any value more than 1 standard deviation from the mean — and print the full list.

Hint: Use `f"{value:,}"` for comma formatting. List comprehension syntax: `[v for v in page_views if v > avg]`. Outlier check: `abs(v - avg) > sd`. Calculate avg and sd before the comprehension.

Expected Output

```
Exercise 6 -- Website Traffic Spread Analysis
30-day page views tracked.
```

```
Best day:      2,456 views
Worst day:     876 views
Range:         1,580 views
Mean/day:      1,379 views
Median/day:    1,345 views
Std Dev:       372 views
```

```
Days above average: 14 (47%)
Days below average: 16 (53%)
```

```
High/low traffic outlier days: [2341, 876, 2100, 890, 1890, 1789, 1890, 2456]
```

Write your code here:

SCREENSHOT 6

Screenshot the terminal showing all traffic metrics and the outlier days list. Paste below.

Analyst Reflection

Q1 Why is range alone an incomplete picture of traffic variability? What additional statistic fills in the gap?

Answer:

Q2 You found outlier days with unusually high or low traffic. How would you investigate why those days occurred? What business questions would you ask?

Answer:

Q3 If writing a monthly report for the marketing director, which three statistics from Exercise 6 would you highlight, and how would you phrase each in plain English?

Answer:

Ex. 7 Third-Party Libraries — statistics vs numpy

[Intermediate]

Analyst Scenario: Your manager says "run this in numpy." You've never used it. Install it, import it, and write a script that compares it to the built-in statistics module on the same dataset.

Your Task

1. In the VS Code Terminal (NOT your .py file), run: `pip install numpy`. Then run `pip list` and confirm numpy appears.
2. In your .py file, create `data = [23.4, 25.1, 22.8, 26.3, 24.5, 23.9, 25.7, 24.1, 23.6, 25.2]`.
3. Using the statistics module, print mean and stdev rounded to 4 decimal places.
4. Wrap a numpy import in a try/except ImportError block. If numpy is installed, print `np.mean()`, `np.std()` (default `ddof=0`), and `np.std(ddof=1)`. If not installed, print a helpful error message.

Hint: try: / import numpy as np / except ImportError: / print("not installed"). `statistics.stdev()` is sample std dev (`ddof=1`). numpy's `np.std()` defaults to population (`ddof=0`). Use `ddof=1` to match.

Expected Output

Exercise 7 -- Standard Library vs Third-Party Libraries

Using Python statistics module (standard library -- no install):

```
statistics.mean() = 24.5400
```

```
statistics.stdev() = 1.1165
```

Using numpy (third-party -- must be installed):

```
np.mean() = 24.5400
```

```
np.std() = 1.0597 (population stdev, ddof=0)
```

```
np.std(ddof=1) = 1.1165 (matches statistics.stdev -- sample stdev)
```

Write your code here:

SCREENSHOT 7

Screenshot the terminal showing (1) the pip install output and (2) the Exercise 7 library comparison output. Paste below.

Analyst Reflection

Q1 What is the difference between a standard library module and a third-party library? Give two reasons you might prefer statistics over numpy for a given task.

Answer:

Q2 Why is the try/except ImportError pattern useful in professional analytics scripts? Give another situation where you would use try/except in data work.

Answer:

Q3 What is the difference between sample standard deviation (ddof=1) and population standard deviation (ddof=0)? Which should an analyst use and when?

Answer:

Ex. 8 Multi-Dataset Comparison — A/B Test**[Intermediate]**

Analyst Scenario: The product team ran a 20-day A/B test on two checkout button colors. You must compare performance, calculate lift, and make a data-driven recommendation.

Your Task

1. Create version_A = [3.2,3.8,4.1,3.5,3.9,4.2,3.7,4.0,3.6,3.8,4.3,3.9,3.7,4.1,3.5,3.8,4.0,3.6,3.9,4.2] and version_B = [4.1,5.2,3.8,4.9,5.1,4.3,4.8,5.0,4.2,4.9,3.9,5.3,4.6,4.8,4.1,5.0,4.7,4.9,4.3,5.1].
2. Loop over both datasets as (name, data) tuples. For each, print: mean, median, std dev, min, and max — formatted to 2 decimal places.
3. After the loop, calculate lift: $((\text{mean_B} - \text{mean_A}) / \text{mean_A}) * 100$. Print it rounded to 1 decimal.
4. Print a recommendation: if mean_B > mean_A, recommend Version B with the lift %. Otherwise recommend keeping Version A.

Hint: Store datasets as a list of tuples: datasets = [("Version A (Blue)", version_A), ...]. Lift formula: $((\text{new} - \text{old}) / \text{old}) * 100$. Use `:.2f` for 2 decimal places and `:.1f` for 1.

Expected Output

Exercise 8 -- A/B Test Comparison

```
=== Version A (Blue Button) ===
Mean conversion:   3.89%
Median conversion: 3.90%
Std Dev:          0.27%
Min: 3.20%   Max: 4.30%
```

```
=== Version B (Green Button) ===
Mean conversion:   4.70%
Median conversion: 4.85%
Std Dev:          0.44%
Min: 3.80%   Max: 5.30%
```

Lift from A to B: +20.8%

Recommendation: Launch Version B. It converts 20.8% better on average.

Write your code here:

**SCREENSHOT
8**

Screenshot the terminal showing both version comparisons and the analyst recommendation. Paste below.

Analyst Reflection

Q1 Version B has a higher mean but also a higher standard deviation. As an analyst, does that concern you? What does higher variability mean for the business?

Answer:

Q2 What is "lift" in an A/B test context? Why do analysts express improvement as a percentage rather than an absolute number?

Answer:

Q3 Descriptive statistics alone cannot prove Version B is definitively better — why not? What additional analysis would a real analyst perform before a final recommendation?

Answer:

Ex. 9

Grouped Statistics — Salary Equity

[Advanced]

Analyst Scenario: HR asks you to examine whether salaries are equitable across departments. You must produce a formatted comparison table and identify the pay gap.

Your Task

1. Create a departments dictionary: Engineering: [95,102,88,110,97,105,91,99,108,94], Marketing: [62,68,71,65,73,60,69,74,67,70], Operations: [55,58,61,57,63,56,59,62,58,60], Data Analytics: [78,85,92,81,88,76,90,83,87,80]. All values in \$k.
2. Print a header row with column labels (Department, Mean, Median, StdDev, Min, Max) separated by a divider line.
3. Loop over the dictionary. For each department, store the mean in a dept_means dict and print a formatted row using f-string column alignment (<18 for name, >7.1f for numbers).
4. Flatten all salaries into one list and print company-wide stats. Use max(dept_means, key=dept_means.get) and min() to find the highest and lowest-paid departments, then print the pay gap.

Hint: Flatten nested lists: [s for sal_list in departments.values() for s in sal_list]. F-string alignment: f"{name:<18} \${avg:>7.1f}k". max(dict, key=dict.get) returns the key with the highest value.

Expected Output

```
Exercise 9 -- Department Salary Analysis
-----
Department      Mean      Median   StdDev   Min      Max
-----
Engineering      $99.9k    $98.5k   $7.2k    $88k     $110k
Marketing         $67.9k    $68.0k   $4.6k    $60k     $74k
Operations        $58.9k    $58.5k   $2.6k    $55k     $63k
Data Analytics    $84.0k    $84.0k   $5.4k    $76k     $92k
-----
Company-wide      $77.7k    $78.5k   $17.6k

Highest avg salary: Engineering ($99.9k)
```

Lowest avg salary: Operations (\$58.9k)
Pay gap between highest and lowest department: \$41.0k

Write your code here:

SCREENSHOT 9

Screenshot the terminal showing the full salary comparison table and the pay gap. Paste below.

Analyst Reflection

Q1 Which department shows the highest internal pay variation (std dev)? What business factors might cause salaries to vary more within one department than another?

Answer:

Q2 The company-wide standard deviation is much larger than any single department's. Why? What does this tell you about analyzing data at different levels of aggregation?

Answer:

Q3 Write 2–3 sentences as if presenting to the HR Director. Which statistics from your output would you highlight and what action would you recommend?

Answer:

Ex. 10

Full Analytics Pipeline — E-Commerce Report

[Advanced]

Analyst Scenario: Build a complete, structured descriptive analytics report from raw e-commerce order data — exactly what a real analyst would deliver to company leadership.

Your Task

1. Use a from...import statement to import mean, median, stdev, variance, and multimode from statistics. Create orders =
[34.99,129.50,45.00,89.99,210.00,34.99,67.50,149.99,22.00,399.99,55.75,34.99,88.00,175.00,45.00,33.50,250.00,78.25,34.99,112.00,65.00,45.00,189.99,29.99,55.75,99.00,34.99,145.00,67.50,315.00,45.00,88.00,34.99,55.75,200.00,79.99,45.00,125.00,34.99,98.50].
2. Before printing, compute all values: total revenue (sum), order count (len), mean, median, multimode, stdev, variance, range, min, max.
3. Print a formatted OVERVIEW section (total orders, total revenue) and CENTRAL TENDENCY section (mean, median, most common values).
4. Print a SPREAD & VARIABILITY section (stdev, variance, range, min, max) and an ORDER SIZE SEGMENTS section using list comprehensions for small (<\$50), medium (\$50–\$149), and large (\$150+) orders — each showing count, %, and avg.
5. Print an ANALYST INSIGHTS section: auto-detect if mean > median (print how much and recommend reporting median). If 40%+ of orders are small, print an upsell recommendation. Wrap the entire output with "=" * 55 borders.

Hint: Calculate everything first, then print. Use sum() for revenue, multimode() returns a list. Segments: [o for o in orders if o < 50]. Percentage: len(small)/len(orders)*100. Format currency with \${value:.2f}.

Expected Output

```
=====
E-COMMERCE ORDER VALUE -- DESCRIPTIVE ANALYTICS
=====
OVERVIEW
  Total Orders:      40
  Total Revenue:    $4,677.59
CENTRAL TENDENCY
  Mean Order Value:  $116.94
  Median Order Val:  $83.13
  Most Common ($):   ['$34.99']
SPREAD & VARIABILITY
  Std Deviation:     $88.23
  Variance:          $7784.38
  Range:             $377.99
  Min Order:         $22.00
  Max Order:         $399.99
ORDER SIZE SEGMENTS
  Small  (under $50):  15 orders  (38%)  Avg: $37.11
  Medium ($50-$149):  17 orders  (43%)  Avg: $80.99
  Large  ($150+):      8 orders  (20%)  Avg: $261.18
ANALYST INSIGHTS
  Mean exceeds median by $33.81 -- large orders skew the average.
  Recommendation: Report MEDIAN as the typical order value.
=====
```

Write your code here:

SCREENSHOT 10

Take a final screenshot showing the COMPLETE analytics report output from top to bottom. This is your capstone — make sure all 5 sections are visible. Paste below.

Analyst Reflection

Q1 The Analyst Insights section auto-detects when mean > median and prints a recommendation. Why is automating this kind of interpretation valuable in a real analytics pipeline?

Answer:

Q2 Looking at the order size segmentation, what business strategies might you recommend based on the percentage breakdown between small, medium, and large orders?

Answer:

Q3 Exercise 10 is called a "pipeline." What does that term mean in data analytics, and how does your script demonstrate the concept? What would the next steps be?

Answer:

Part D

The Data Analyst Mindset — Capstone Reflection

Answer these broader questions after completing all 10 exercises. They reflect how professional data analysts think.

M1 The statistics module is useful but professional analysts typically use pandas and numpy for larger datasets. Based on what you learned, what are two specific limitations of the statistics module that would push you toward a third-party library?

Answer:

M2 You computed mean, median, mode, standard deviation, variance, and range across multiple exercises. If you could show only ONE statistic to a non-technical business executive, which would you choose and why? Does your answer change depending on the dataset?

Answer:

M3 In Exercise 3 (salaries) and Exercise 6 (web traffic) you identified outliers. Describe a systematic process — in plain English — a data analyst should follow when discovering an outlier. What questions do you ask? What decisions do you make?

Answer:

M4 All 10 exercises used simulated data. List three challenges you would face working with REAL data from a CSV file or database. How would your Python code need to change?

Answer:

M5 Reflect on the three import styles you used. Write a short paragraph explaining which style you would adopt as your personal standard and why — as if writing a coding style guide for a new teammate.

Answer:

Part E Submission Checklist

Check every box before submitting. Each item must be complete.

☐	File saved as analytics_practice_1_2.py (confirm the tab name in VS Code)
☐	Header comment includes your Name, Date, and Course
☐	All 10 exercises written from scratch — code not copied from the Solutions section
☐	Each exercise separated by a comment label (e.g., # -- Exercise 1 -----)
☐	File runs without any errors (no red text in the terminal)
☐	Screenshot A — VS Code showing filename and complete header comment

[]	Screenshot 1 — Three import styles all printing the same mean result
[]	Screenshot 2 — Mean, median, mode for sales data with the Insight line
[]	Screenshot 3 — Both salary tables showing the outlier impact on mean vs median
[]	Screenshot 4 — Mode, text bar chart of ratings, most ordered size
[]	Screenshot 5 — Both warehouse stats and the variability ratio
[]	Screenshot 6 — All traffic metrics, above/below average counts, outlier list
[]	Screenshot 7 — pip install output and statistics vs numpy comparison
[]	Screenshot 8 — A/B test stats for both versions, lift %, recommendation
[]	Screenshot 9 — Formatted department salary table and pay gap
[]	Screenshot 10 — Complete e-commerce analytics report (all 5 sections visible)
[]	All Analyst Reflection questions answered in full sentences
[]	All Mindset questions (M1–M5) answered in full sentences

Part
F

Statistics Library Quick Reference

Function	Syntax	Returns	Use When
statistics.mean()	mean(data)	Float — average	Symmetric data, no extreme outliers
statistics.median()	median(data)	Middle value	Skewed data or outliers present
statistics.mode()	mode(data)	Single most common	Most frequent value; categorical data
statistics.multimode()	multimode(data)	List of all modes	When ties exist or all peaks needed
statistics.stdev()	stdev(data)	Sample std dev	Measuring spread; comparing groups
statistics.variance()	variance(data)	Sample variance	When variance needed for calculations
max(data) / min(data)	max(data)	Largest / smallest	Finding extremes (built-in, no import)
len(data)	len(data)	Count of items	Counting observations (built-in)
sum(data)	sum(data)	Total of all values	Summing all values (built-in)

Pro Tip: Always import only what you need. In a small analytics script, `from statistics import mean, median, stdev` is cleaner than `import statistics` if you only use three functions. In larger projects, the full import keeps your namespace clear and shows exactly where each function comes from.

SOLUTIONS

Class Practice Exercise 1.2 | Instructor / Self-Check Reference

Attempt every problem on your own FIRST. Only use this section to verify your output.

Sol 1 Exploring Import Styles

```
# -- Exercise 1: Import Styles -----
import statistics
import math
from statistics import mean, median
import statistics as stats

print("Exercise 1 -- Import Styles")
print("math.pi =", math.pi)
print("sqrt(144) =", math.sqrt(144))
print("Mean (full module): ", statistics.mean([10, 20, 30]))
print("Mean (from import): ", mean([10, 20, 30]))
print("Mean (alias stats): ", stats.mean([10, 20, 30]))
print("All three return the same result -- different styles, same function!")
```

Why it works: Three import styles exist: full module (`statistics.mean`), selective import (`from statistics import mean` — no prefix needed), and alias (`import statistics as stats` — shorter prefix). All call the same underlying function; they differ only in how you reference it in your code.

Sol 2 Central Tendency — Monthly Sales

```
# -- Exercise 2: Central Tendency -- Sales Data -----
import statistics

monthly_sales = [42, 55, 61, 48, 73, 58, 66, 51, 70, 63, 55, 80]
sales_mean = statistics.mean(monthly_sales)
sales_median = statistics.median(monthly_sales)
sales_mode = statistics.mode(monthly_sales)

print("Exercise 2 -- Sales Central Tendency")
print(f"Dataset: {monthly_sales}")
print(f"Count: {len(monthly_sales)} months")
print(f"Mean: ${sales_mean:.2f}k (average monthly sales)")
print(f"Median: ${sales_median}k (middle-ranked month)")
print(f"Mode: ${sales_mode}k (most frequent sales figure)")

if sales_mean > sales_median:
    print("Insight: Mean > Median -- a high-sales month is pulling the average up.")
elif sales_mean < sales_median:
    print("Insight: Mean < Median -- a low-sales month is dragging the average down.")
else:
    print("Insight: Mean equals Median -- data is likely symmetric.")
```

Why it works: `statistics.mean()`, `statistics.median()`, and `statistics.mode()` each accept a list directly. The `if/elif/else` block automates the analyst insight by comparing mean and median: when `mean > median`, at least one high value is pulling the average upward, making the median a better measure of a "typical" month.

Sol 3 Outlier Effect — Employee Salaries

```
# -- Exercise 3: Outlier Effect -- Salary Analysis -----
import statistics

salaries_no_outlier = [45000, 48000, 52000, 47000, 51000, 49000, 53000]
salaries_with_outlier = [45000, 48000, 52000, 47000, 51000, 49000, 53000, 420000]

print("Exercise 3 -- Outlier Effect on Central Tendency")
print()
print("--- Without Executive Salary ---")
print(f"Mean:    ${statistics.mean(salaries_no_outlier):,.0f}")
print(f"Median:  ${statistics.median(salaries_no_outlier):,.0f}")
print()
print("--- With Executive Salary ---")
print(f"Mean:    ${statistics.mean(salaries_with_outlier):,.0f}")
print(f"Median:  ${statistics.median(salaries_with_outlier):,.0f}")

mean_diff = statistics.mean(salaries_with_outlier) -
statistics.mean(salaries_no_outlier)
median_diff = statistics.median(salaries_with_outlier) -
statistics.median(salaries_no_outlier)
print(f"Adding the outlier raised the mean by:    ${mean_diff:,.0f}")
print(f"Adding the outlier changed the median by: ${median_diff:,.0f}")
print()
print("Analyst Decision: Report the MEDIAN when outliers are present.")
```

Why it works: The `{:,0f}` f-string format adds commas and removes decimals for large numbers. The mean is the sum divided by count — one extreme value dramatically shifts the total. The median is just the middle-ranked value; adding one outlier at the end shifts it by only one position. This is why median is preferred for skewed distributions like salaries or house prices.

Sol 4 Mode Analysis — Customer Survey

```
# -- Exercise 4: Mode Analysis -- Survey Data -----
import statistics

ratings = [4, 5, 3, 4, 5, 4, 2, 4, 5, 3, 4, 1, 4, 5, 3, 4, 4, 5, 2, 4]
sizes_ordered = ["M", "L", "M", "S", "M", "XL", "L", "M", "S", "M", "L", "M"]

print("Exercise 4 -- Mode Analysis")
print(f"Most common rating: {statistics.mode(ratings)} stars")
print(f"Total responses: {len(ratings)}")
print()

for star in [1, 2, 3, 4, 5]:
    count = ratings.count(star)
    bar = "#" * count
    print(f" {star} stars: {bar} ({count} responses)")

print()
print(f"Most ordered size: {statistics.mode(sizes_ordered)}")
all_modes = statistics.multimode(ratings)
print(f"All modes (ties included): {all_modes}")
```

Why it works: `list.count(value)` counts how many times a value appears — efficient for small lists. `"#" * n` uses Python string multiplication to repeat a character, creating a text bar chart. `statistics.mode()` returns

one value (raises an error on ties in Python 3.7); `statistics.multimode()` always returns a list of all most-frequent values and never raises errors on ties.

Sol 5 Standard Deviation — Warehouse Temperatures

```
# -- Exercise 5: Standard Deviation & Variance -----
import statistics

warehouse_A = [68, 70, 69, 71, 68, 70, 69, 70, 68, 71]
warehouse_B = [58, 82, 61, 79, 55, 83, 64, 78, 60, 80]

print("Exercise 5 -- Standard Deviation & Variance")
print()

for name, data in [("Warehouse A", warehouse_A), ("Warehouse B", warehouse_B)]:
    avg = statistics.mean(data)
    sd = statistics.stdev(data)
    var = statistics.variance(data)
    rng = max(data) - min(data)
    print(f"--- {name} ---")
    print(f"  Temperatures: {data}")
    print(f"  Mean:      {avg:.1f} F")
    print(f"  Std Dev:   {sd:.2f} F")
    print(f"  Variance:  {var:.2f}")
    print(f"  Range:    {rng} F   (min={min(data)}, max={max(data)})")
    print()

sdA = statistics.stdev(warehouse_A)
sdB = statistics.stdev(warehouse_B)
print(f"Warehouse B has {sdB/sdA:.1f}x more variability -- same average, very different spread.")
```

Why it works: Packing both datasets into a list of tuples lets one loop handle both without code repetition. `statistics.stdev()` computes sample standard deviation. Range = max - min (a simple but incomplete spread measure). The ratio `sdB/sdA` quantifies exactly how many times more variable one dataset is than another — a clear number to give a non-technical manager.

Sol 6 Range & Spread — Website Traffic

```
# -- Exercise 6: Range, Min, Max, Outlier Detection -----
import statistics

page_views = [1203, 987, 1450, 1102, 876, 2341, 1567, 1234, 905, 1678,
              1345, 2100, 1456, 890, 1234, 1567, 1890, 1023, 1345, 1678,
              934, 1123, 1456, 1789, 1012, 1345, 1567, 1890, 1234, 2456]

avg = statistics.mean(page_views)
sd = statistics.stdev(page_views)

print("Exercise 6 -- Website Traffic Spread Analysis")
print("30-day page views tracked.")
print()
print(f"Best day:      {max(page_views):,} views")
print(f"Worst day:     {min(page_views):,} views")
print(f"Range:         {max(page_views) - min(page_views):,} views")
print(f"Mean/day:      {avg:,.0f} views")
```

```

print(f"Median/day:  {statistics.median(page_views):,.0f} views")
print(f"Std Dev:     {sd:,.0f} views")
print()

above_avg = [v for v in page_views if v > avg]
below_avg = [v for v in page_views if v < avg]
print(f"Days above average: {len(above_avg)} ({len(above_avg)/30*100:.0f}%)")
print(f"Days below average: {len(below_avg)} ({len(below_avg)/30*100:.0f}%)")
print()

outlier_days = [v for v in page_views if abs(v - avg) > sd]
print(f"High/low traffic outlier days: {outlier_days}")

```

Why it works: List comprehensions concisely filter a list: [v for v in page_views if condition]. Compute avg and sd before the comprehensions since both are used multiple times. `abs(v - avg) > sd` identifies values more than one standard deviation away from the mean — a common first-pass outlier detection rule. `{:,}` adds comma separators to integers for readability.

Sol 7 Third-Party Libraries — statistics vs numpy

```

# -- Exercise 7: Standard Library vs Third-Party Libraries -----
import statistics

data = [23.4, 25.1, 22.8, 26.3, 24.5, 23.9, 25.7, 24.1, 23.6, 25.2]

print("Exercise 7 -- Standard Library vs Third-Party Libraries")
print()
print("Using Python statistics module (standard library -- no install):")
print(f"  statistics.mean()   = {statistics.mean(data):.4f}")
print(f"  statistics.stdev()    = {statistics.stdev(data):.4f}")
print()

try:
    import numpy as np
    print("Using numpy (third-party -- must be installed):")
    print(f"  np.mean()              = {np.mean(data):.4f}")
    print(f"  np.std()                = {np.std(data):.4f} (population stdev, ddof=0)")
    print(f"  np.std(ddof=1)          = {np.std(data, ddof=1):.4f} (matches")
    print(f"  statistics.stdev())")
except ImportError:
    print("numpy not installed -- run: pip install numpy")
    print("This is why knowing pip is essential for data analysts!")

```

Why it works: The try/except ImportError pattern makes scripts resilient — they run whether or not a library is installed, with a helpful message if it is missing. `statistics.stdev()` uses `ddof=1` (sample standard deviation, divides by $n-1$, used when your data is a sample from a larger population). `numpy's np.std()` defaults to `ddof=0` (population standard deviation, divides by n). Use `ddof=1` for sample data, `ddof=0` only when you have the full population.

Sol 8 Multi-Dataset Comparison — A/B Test

```

# -- Exercise 8: A/B Test Comparison -----
import statistics

version_A = [3.2, 3.8, 4.1, 3.5, 3.9, 4.2, 3.7, 4.0, 3.6, 3.8,
              4.3, 3.9, 3.7, 4.1, 3.5, 3.8, 4.0, 3.6, 3.9, 4.2]

```

```

version_B = [4.1,5.2,3.8,4.9,5.1,4.3,4.8,5.0,4.2,4.9,
             3.9,5.3,4.6,4.8,4.1,5.0,4.7,4.9,4.3,5.1]

print("Exercise 8 -- A/B Test Comparison")
print()

datasets = [("Version A (Blue Button)", version_A),
            ("Version B (Green Button)", version_B)]

for name, data in datasets:
    print(f"=== {name} ===")
    print(f"  Mean conversion:  {statistics.mean(data):.2f}%")
    print(f"  Median conversion: {statistics.median(data):.2f}%")
    print(f"  Std Dev:          {statistics.stdev(data):.2f}%")
    print(f"  Min: {min(data):.2f}%    Max: {max(data):.2f}%")
    print()

mean_A = statistics.mean(version_A)
mean_B = statistics.mean(version_B)
lift = ((mean_B - mean_A) / mean_A) * 100
print(f"Lift from A to B: +{lift:.1f}%")

if mean_B > mean_A:
    print(f"Recommendation: Launch Version B. It converts {lift:.1f}% better on average.")
else:
    print("Recommendation: Keep Version A. No improvement detected.")

```

Why it works: Storing datasets as (name, data) tuples in a list enables one clean loop to process both without repetition. Lift = (new - baseline) / baseline * 100 — a percentage improvement over the control. Note that Version B's higher standard deviation means its results are less consistent day-to-day; a real analyst would also run a statistical significance test (e.g., t-test) before launching.

Sol 9 Grouped Statistics — Salary Equity

```

# -- Exercise 9: Grouped Statistics -- Department Salaries -----
import statistics

departments = {
    "Engineering": [95,102,88,110,97,105,91,99,108,94],
    "Marketing":   [62,68,71,65,73,60,69,74,67,70],
    "Operations":  [55,58,61,57,63,56,59,62,58,60],
    "Data Analytics": [78,85,92,81,88,76,90,83,87,80]
}

print("Exercise 9 -- Department Salary Analysis")
print("-" * 60)
print(f'{"Department":<18} {"Mean":>8} {"Median":>8} {"StdDev":>8} {"Min":>6} {"Max":>6} ')
print("-" * 60)

dept_means = {}
for dept, salaries in departments.items():
    avg = statistics.mean(salaries)
    med = statistics.median(salaries)
    sd  = statistics.stdev(salaries)
    dept_means[dept] = avg
    print(f'{dept:<18} ${avg:>7.1f}k ${med:>6.1f}k ${sd:>6.1f}k ${min(salaries):>5}k ${max(salaries):>5}k')

```

```

print("-" * 60)
all_salaries = [s for sal_list in departments.values() for s in sal_list]
print(f'{"Company-wide":<18} ${statistics.mean(all_salaries):>7.1f}k'
      ${statistics.median(all_salaries):>6.1f}k ${statistics.stdev(all_salaries):>6.1f}k')
print()

top_dept = max(dept_means, key=dept_means.get)
low_dept = min(dept_means, key=dept_means.get)
print(f"Highest avg salary: {top_dept} (${dept_means[top_dept]:.1f}k)")
print(f"Lowest avg salary: {low_dept} (${dept_means[low_dept]:.1f}k)")
print(f"Pay gap between highest and lowest department: ${dept_means[top_dept]-
dept_means[low_dept]:.1f}k")

```

Why it works: F-string alignment: `{:<18}` left-aligns in 18 characters; `{:>7.1f}` right-aligns a float with 1 decimal in 7 characters. The nested list comprehension `[s for sal_list in d.values() for s in sal_list]` flattens a dict of lists into one flat list. `max(dict, key=dict.get)` finds the dictionary key whose value is largest — a clean Pythonic pattern for ranked lookups.

Sol 10 Full Analytics Pipeline — E-Commerce Report

```

# -- Exercise 10: Full Analytics Pipeline -----
from statistics import mean, median, stdev, variance, multimode

orders = [
    34.99, 129.50, 45.00, 89.99, 210.00, 34.99, 67.50, 149.99,
    22.00, 399.99, 55.75, 34.99, 88.00, 175.00, 45.00, 33.50,
    250.00, 78.25, 34.99, 112.00, 65.00, 45.00, 189.99, 29.99,
    55.75, 99.00, 34.99, 145.00, 67.50, 315.00, 45.00, 88.00,
    34.99, 55.75, 200.00, 79.99, 45.00, 125.00, 34.99, 98.50
]

# Compute everything before printing
total_rev = sum(orders)
avg_order = mean(orders)
med_order = median(orders)
common    = multimode(orders)
sd_order  = stdev(orders)
var_order = variance(orders)
rng_order = max(orders) - min(orders)
small     = [o for o in orders if o < 50]
medium    = [o for o in orders if 50 <= o < 150]
large     = [o for o in orders if o >= 150]

print("=" * 55)
print("  E-COMMERCE ORDER VALUE -- DESCRIPTIVE ANALYTICS")
print("=" * 55)
print("\nOVERVIEW")
print(f"  Total Orders:      {len(orders)}")
print(f"  Total Revenue:      ${total_rev:,.2f}")
print("\nCENTRAL TENDENCY")
print(f"  Mean Order Value:    ${avg_order:.2f}")
print(f"  Median Order Val:    ${med_order:.2f}")
print(f"  Most Common ($):     {[f'${v}' for v in common]}")
print("\nSPREAD & VARIABILITY")
print(f"  Std Deviation:       ${sd_order:.2f}")
print(f"  Variance:            ${var_order:.2f}")
print(f"  Range:               ${rng_order:.2f}")
print(f"  Min Order:           ${min(orders):.2f}")

```

```

print(f"  Max Order:           ${max(orders):.2f}")
print("\nORDER SIZE SEGMENTS")
for label, seg in [("Small   (under $50)", small),
                  ("Medium ($50-$149)", medium),
                  ("Large   ($150+)", large)]:
    pct = len(seg)/len(orders)*100
    print(f"  {label}: {len(seg):>4} orders  ({pct:.0f}%)  Avg: ${mean(seg):.2f}")
print("\nANALYST INSIGHTS")
if avg_order > med_order:
    diff = avg_order - med_order
    print(f"  Mean exceeds median by ${diff:.2f} -- large orders skew the average.")
    print("  Recommendation: Report MEDIAN as the typical order value.")
if len(small)/len(orders)*100 > 40:
    pct_s = len(small)/len(orders)*100
    print(f"  {pct_s:.0f}% of orders under $50 -- consider bundling or upsell prompts.")
print("=" * 55)

```

Why it works: A pipeline is a sequence of steps that transforms raw input into structured output: ingest data -> compute statistics -> segment -> interpret -> print report. Computing all values before any print() calls is clean pipeline design. The from...import style documents exactly which functions the script depends on. The ANALYST INSIGHTS block shows how Python can encode business logic, not just arithmetic — making the script genuinely useful to a stakeholder, not just technically correct.